Docker & Kubernetes for Developers

Session 03: Pod Yönetimi ve Deployment Stratejileri

Eğitmen: Hüseyin Babal
Süre: 2 Saat (Teori + Hands-on Lab)

📋 Bu Oturum İçeriği

- ✅ Pod lifecycle ve yönetimi
- ✅ ReplicaSet kavramı ve kullanımı
- ✅ Deployment objesi ve özellikleri
- ✅ Rolling update stratejileri
- ✅ Rollback işlemleri
- ✅ **E-commerce microservices deployment (Lab)**
- ✅ Health checks ve probes

🔄 Pod Lifecycle

```
┌─────────┐   ┌─────────┐   ┌─────────┐
│ Pending │ -> │ Running │ -> │Succeeded│
└─────────┘   └─────────┘   │   or    │
                            │Running  │
                            └─────────┘

                            ┌─────────┐
                            │ Failed  │
                            └─────────┘
```

Pod Phases:

- **Pending:** Scheduling bekliyor
- **Running:** En az 1 container çalışıyor
- **Succeeded:** Tüm container'lar başarıyla bitti
- **Failed:** En az 1 container hata verdi
- **Unknown:** Pod durumu belirlenemiyor

❌ Pod Limitations

Problemler:

- **Manuel Pod yönetimi** zahmetli
- **Pod crash** olursa yenisi otomatik oluşmaz
- **Scaling** manuel yapılmalı
- **Rolling updates** imkansız
- **Self-healing** yok

Çözüm:

```
Pod (Raw) ❌  →  ReplicaSet + Deployment ✅
```

🔁 ReplicaSet Nedir?

> **ReplicaSet**, belirtilen sayıda Pod'un her zaman çalışmasını garantiler.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21
```

🎯 ReplicaSet Özellikler

Self-Healing

```
kubectl delete pod nginx-replicaset-abc123
# Yeni pod otomatik oluşur!
kubectl get pods
```

Scaling

```
kubectl scale replicaset nginx-replicaset --replicas=5
kubectl get pods -w
```

Label Selector

- ReplicaSet, label'lar ile Pod'ları takip eder
- Label değişirse ReplicaSet kontrolü kaybeder

🚀 Deployment - Production Ready

> **Deployment**, ReplicaSet'lerin üstünde bir abstraction katmanıdır.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21
        ports:
        - containerPort: 80
```

🌊 Rolling Update Strategy

Default Strategy: RollingUpdate

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 25%    # En fazla %25 pod down
      maxSurge: 25%          # En fazla %25 extra pod
```

Recreate Strategy

```yaml
spec:
  strategy:
    type: Recreate  # Önce tüm pod'ları sil, sonra yenilerini oluştur
```

Production'da RollingUpdate kullanın! ✅

🔄 Rolling Update Demo

```
# Deployment oluştur
kubectl create deployment nginx --image=nginx:1.20

# Rolling update başlat
kubectl set image deployment/nginx nginx=nginx:1.21

# Update sürecini izle
kubectl rollout status deployment/nginx

# Rollout history
kubectl rollout history deployment/nginx
```

Update Süreci:

1. Yeni ReplicaSet oluştur
2. Yeni Pod'ları kademeli olarak başlat
3. Eski Pod'ları kademeli olarak sil
4. Eski ReplicaSet'i scale down yap

⏮ Rollback İşlemleri

Rollback Commands

```
# Önceki versiyona dön
kubectl rollout undo deployment/nginx

# Belirli revision'a dön
kubectl rollout undo deployment/nginx --to-revision=1

# Revision geçmişi
kubectl rollout history deployment/nginx

# Belirli revision detayı
kubectl rollout history deployment/nginx --revision=2
```

Rollback Senaryoları:

- Yeni image'da bug var
- Performance degradation
- Security vulnerability

🏥 Health Checks ve Probes

Liveness Probe - Container yaşıyor mu?

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
```

Readiness Probe - Traffic almaya hazır mı?

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

🔧 Probe Types

HTTP Probe

```yaml
httpGet:
  path: /health
  port: 8080
  httpHeaders:
  - name: Custom-Header
    value: Awesome
```

TCP Probe

```yaml
tcpSocket:
  port: 8080
```

Command Probe

```yaml
exec:
  command:
  - cat
  - /tmp/healthy
```

🏪 Lab: E-commerce User Service Deployment

```yaml
# user-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  labels:
    app: user-service
    component: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
      - name: user-service
        image: nginx:1.21  # Geçici olarak nginx kullanıyoruz
        ports:
        - containerPort: 80
        env:
        - name: SERVICE_NAME
          value: "user-service"
        - name: VERSION
          value: "v1.0.0"
```

🛍️ Lab: Product Service Deployment

```yaml
# product-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
      - name: product-service
        image: nginx:1.21
        ports:
        - containerPort: 80
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "200m"
```

📦 Lab: Order Service Deployment

```yaml
# order-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
      - name: order-service
        image: nginx:1.21
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
```

🧪 Lab Uygulaması

Adım 1: Deployment'ları Oluştur

```
kubectl apply -f user-service-deployment.yaml
kubectl apply -f product-service-deployment.yaml
kubectl apply -f order-service-deployment.yaml
```

Adım 2: Durumu Kontrol Et

```
kubectl get deployments
kubectl get replicasets
kubectl get pods -l app=user-service
```

Adım 3: Scale Test

```
kubectl scale deployment user-service --replicas=5
kubectl get pods -w
```

🔄 Rolling Update Lab

Image Update Test

```
# User service image'ı güncelle
kubectl set image deployment/user-service user-service=nginx:1.22

# Update sürecini izle
kubectl rollout status deployment/user-service

# Pod'ların değişimini gözlemle
kubectl get pods -l app=user-service -w
```

Rollback Test

```
# Rollback yap
kubectl rollout undo deployment/user-service

# History kontrol
kubectl rollout history deployment/user-service
```

📝 Sonraki Oturum Önizlemesi

Session 04: Kubernetes Hizmet Keşfi ve Ağ Yapısı

- **Service** types (ClusterIP, NodePort, LoadBalancer)
- **Ingress** controllers ve routing
- **Network policies** ve güvenlik
- **Service mesh** giriş
- **E-commerce services arası iletişim kurma**

Hazırlık:

- Deployment kavramlarını pekiştirin
- Network temellerini gözden geçirin

🙏 Teşekkürler!

Deployment Master! 🎉

Bugün öğrendiklerimiz:
✅ ReplicaSet ve Deployment'ı öğrendik
✅ Rolling updates yaptık
✅ E-commerce microservices deploy ettik
✅ Health checks yapılandırdık

İletişim:

- GitHub: @huseyinbabal

Sonraki Oturum: Session 04 - Kubernetes Hizmet Keşfi ve Ağ Yapısı